

Compiler Design

Janis Hutz
<https://janishutz.com>

September 21, 2026



“A funny quote by a professor”

- Prof. Dr. Professor Name, YEAR

HS2026, ETHZ
Summary of the Script and Lectures

Contents

1	Introduction	2
2	OCAML	3
2.1	The basics	3
3	x86Lite	4
4	LLVM	4
5	Parsing	4
6	Lexing	4
7	Optimizations	4

1 Introduction

This summary is still a very early version and structure and content will still change and errors will be fixed.

2 OCAML

2.1 The basics

OCAML is a compiled, functional programming language and a descendant of the Meta Language programming language. In contrast to other functional programming languages (such as Haskell), it is actually used in the wild by more than two people.

File: code/ocaml/00_basics.ml

```

1  (* comments are like this and can be nested *)
2  let a_function (a : int) (b : int) : int = a * b
3
4  (* We can use the following to print ANYTHING upon execution *)
5  let () = Printf.printf "Format String (just like C's printf)"
6
7  (* We can also use ; to sepearate statements. The last statement's value is returned
8     Furthermore, this definition of `a_function` shadows the previous one *)
9  let a_function (a : int) : int =
10     Printf.printf "Print anything here";
11     a * a
12
13  let vars_in_func (a : int) =
14     let x = a_function a in
15     Printf.printf "Value is %d" x
16
17  (* This is a simple example of a generic funciton and pattern matching *)
18  let is_empty (a : 'a list) : bool =
19     match a with
20     | [] -> true
21     | _ :: _ -> false (* The underscores are, just like in Haskell, wildcards *)
22
23  let if_func (v1 : string) (v2 : string) : string =
24     if v1 = v2 then "match" else "no match"
25
26  (* Recursive function *)
27  let rec double_all (arg : int list) : int list =
28     match arg with [] -> [] | a :: al -> (a * a) :: double_all al
29
30  (* Recursive Tail function. This is more efficient than the above because it's compiled
31     to a simple loop. Furthermore, the fold functions behave in this way *)
32  let double_all_t (arg : int list) : int list =
33     let rec aux a acc = match a with [] -> acc | v :: vl -> aux vl (v :: acc) in
34     List.rev (aux arg [])
35
36  let print_list (list : 'a list) = List.iter (Printf.printf "%d") list

```

OCAML is dynamically typed (i.e. types don't *have* to be annotated), but they can be, if you so choose. As with Python and all other languages where type annotation is optional, you are strongly encouraged to annotate types, especially for more complex expressions, as that can massively reduce the likeliness of errors.

The first graded assignment's first 300 ish lines also give you a very neat overview over how OCAML works, so absolutely read through those.

For Printf, the format string is equivalent to that of C, a solid cheat sheet can be found [here](#)

3 x86Lite

4 LLVM

5 Parsing

6 Lexing

7 Optimizations